通用长报告Agent系统设计

https://gemini.google.com/app/e3be7e85ca0e2817

User prompt: 当前我需要开发一个分析长报告生成的项目，覆盖多个场景，比如预算报经营分析报告，财务公司尽调报告风险分析尽调报告，人力部门薪酬管理报告分析等等，输入都是用户上传文档，根据不同场景的分析要求，输出最终的分析报告场景，需要支持后续材料补充，多轮交付方式场景，制定章节修改等，但是每次的对话都需要输出最终的长文档报告，我希望设计成一个通用的长文档报告项目，不同场景使用配置文件进行加载，交付方式是想以skills和插件或者外部工具的方式进行交付，在codex或者claude code和deepseekharness这种Agent上进行验证交付。我应该怎么设计这个项目

Response: 设计一套跨场景、支持多轮增量演进、且**每轮对话都能完整输出最终长文档**的智能报告系统，核心矛盾在于：**大模型的单次输出 Token 限制及“惰性改写幻觉”（改一章丢三章）与“每次必须交付全量长文档”之间的冲突。**

针对 **Claude Code**、**Codex**、**DeepSeek Harness** 这类具备本地文件工作区与工具调用能力的 Agent 平台，最佳实践是采用 **“薄 Skill（认知与编排层）+ 状态化外部工具 CLI/MCP（解析、状态树与拼装层）”** 的双层架构：让大模型只做**局部章节的推理与增量生成**，由外部工具通过 **章节级状态树（Section AST）** 在毫秒级内无损拼接并输出全量长报告。

一、 核心架构设计：为什么采用“Skill + 状态化工具”双层解耦？

如果在多轮对话中直接让大模型在上下文里流式重写几万字的全量报告，会导致未修改章节被模型擅自缩写、财务/薪酬数值在多轮传递中发生漂移，且耗费大量 Token 和时间。

通过将系统拆分为以下四层，可以完美兼顾**通用配置、增量补充、定点修改**与**全量输出**：

架构层级	核心组件	职责边界	解决的核心痛点
1. 场景配置层	`scenarios/*.yaml` + 提示词模板	定义不同业务场景（预算、尽调、薪酬）的目录树、指标口径、计算公式与分析方法论	新场景零代码接入，仅需新增配置文件包
2. 解析与事实层	`ingest_engine` (Docling / Pandas)	解析 PDF/Word/Excel，执行确定性数值计算，构建带溯源 ID 的结构化事实库（Fact Ledger）	杜绝大模型口算财务/薪酬数据幻觉；支持多轮增量追加材料
3. 章节状态机层	`workspace_state` (Section DAG)	将长报告拆分为物理隔离的章节文件，维护章节依赖图（DAG）与脏数据标记（Dirty Flag）	指定修改某章节时，其余章节物理锁定，实现**非目标章节 100% 零篡改**
4. 拼装与交付层	`report_assembler` (CLI / MCP)	每次对话结束前，将更新后的章节与未改动章节缓存合并，渲染并输出完整全量长文档	突破单次生成长度上限，保证**每轮对话均交付完整的最终长报告**

二、 目录结构与通用场景配置设计

为了同时适配 Claude Code、Codex 和 DeepSeek Harness，项目采用统一底层 Python 引擎 + 多平台适配壳（Skill & MCP）的工程结构：

```
long-report-agent/
├── AGENTS.md                       # 供 Codex / DeepSeek Harness 读取的全局智能体工作流规范
├── .claude/
│   └── skills/
│       └── long-report-generator/
│           └── SKILL.md            # 供 Claude Code 自动路由加载的 Skill 指令
├── scenarios/                      # 【配置中心】按场景隔离的配置包
│   ├── budget_analysis/            # 场景1：预算与经营分析报告
│   │   ├── config.yaml             # 章节拓扑、依赖关系、必提指标Schema
│   │   ├── formulas.py             # 确定性计算脚本（如预算达成率、同比/环比、杜邦拆解）
│   │   └── prompts/                # 各章节专属分析框架 Prompt
│   ├── fin_due_diligence/          # 场景2：财务公司尽调与风险分析报告
│   └── hr_compensation/            # 场景3：人力部门薪酬管理分析报告
├── engine/                         # 【核心工具层】可作为 CLI 或 MCP Server 运行
│   ├── cli.py                      # 命令行入口（适配 Codex / Bash 工具调用）
│   ├── mcp_server.py               # MCP 协议入口（适配 Claude Code / 支持 MCP 的 Harness）
│   ├── parser.py                   # 多格式文档解析与增量 Diff
│   ├── state_manager.py            # 章节状态机与依赖传播计算
│   └── assembler.py                # 全量文档拼接、交叉引用校验与 Markdown/Docx 导出
└── .report_workspace/              # 【运行时状态区】每次任务自动维护的本地数据库
    ├── manifest.json               # 当前场景、版本号、已加载材料 Hash 列表
    ├── fact_ledger.json            # 抽取出的结构化事实与计算结果（含文档溯源坐标）
    ├── sections/                   # 独立存储的各章节 Markdown 节点
```

```
    │   ├── 01_exec_summary.md
    │   ├── 02_financial_metrics.md
    │   └── 03_risk_assessment.md
    └── dist/
        └── full_report_latest.md    # 每一轮对话最终合并输出的完整长报告
```

通用场景配置文件（`config.yaml`）设计规范

每个场景通过一份声明式配置控制全流程，核心包含材料清单、确定性指标定义、章节依赖有向无环图（DAG）三部分：

```yaml
scenario_id: "fin_due_diligence"
scenario_name: "财务与风险尽职调查分析报告"
global_constraints:
  citation_required: true           # 强制要求所有数据标注来源 [Doc_ID:Page]
  missing_data_policy: "placeholder" # 缺失数据生成标准占位符【待补充材料：XXX】，严禁编造

# 1. 结构化抽取与工具预计算指标
metrics_schema:
  - id: "asset_liability_ratio"
    name: "资产负债率"
    source_type: "table_extraction"
    validator: "0 <= value <= 2.0"
  - id: "cash_flow_coverage"
    name: "经营现金流债务覆盖率"
    compute_script: "formulas.py:calc_cash_coverage"

# 2. 章节结构与联动依赖关系 (DAG)
sections:
  - id: "sec_01_overview"
    title: "一、 尽调执行摘要与核心风险结论"
    prompt_template: "prompts/01_overview.md"
    depends_on: ["sec_02_financial", "sec_03_risk"] # 当第2、3章修改时，第1章自动标记为需联动更新
    required_facts: ["asset_liability_ratio", "cash_flow_coverage"]

  - id: "sec_02_financial"
    title: "二、 历史财务表现与资产质量穿透分析"
    prompt_template: "prompts/02_financial.md"
    depends_on: []
    required_facts: ["asset_liability_ratio", "balance_sheet_summary"]

  - id: "sec_03_risk"
    title: "三、 或有负债、合规瑕疵与重大风险敞口"
    prompt_template: "prompts/03_risk.md"
    depends_on: []
    required_facts: ["litigation_list", "guarantee_details"]
```

三、 多轮交互与“每次输出全量报告”的状态流转机制

为了实现**补充材料、指定章节修改、且每次对话都输出完整长文档**，系统在工作区通过以下四个状态流转阶段运行：

1. 首轮生成（Initial Full Build）

1. **识别场景与初始化**：Agent 根据用户意图调用工具 `init_workspace(scenario="budget_analysis")`，加载对应 `config.yaml` 并在 `.report_workspace/sections/` 下创建章节骨架。
2. **文档解析与预计算**：调用 `ingest_documents(files)`，工具使用 Docling/MarkItDown 将 PDF/Word 转为结构化块，使用 Pandas 解析 Excel 并运行 `formulas.py` 完成严谨的指标计算，写入 `fact_ledger.json`。若发现配置要求的必有数据缺失，自动在对应章节记录 `missing_slots`。
3. **分章撰写与全量拼装**：Agent 按拓扑顺序（先写基础分析章，最后写依赖全局的执行摘要章）调用 `commit_section` 写入各章内容，最后调用 `assemble_full_report()` 将全量报告输出给用户。

2. 后续材料补充（Incremental Material Supplement）

当用户在第 N 轮上传了补充材料（例如：“这是刚拿到的 Q4 补充审计底稿和薪酬明细表”）：

1. **增量解析与影响面分析**：工具对新文档计算 Hash 并提取新事实追加至 `fact_ledger.json`，自动比对哪些章节的 `required_facts` 或 `missing_slots` 被新材料命中。

2. **脏标记传播（Dirty Propagation）**：假设新材料更新了 `sec_03_risk`（第3章风险分析），状态机自动将 `sec_03_risk` 及其上游依赖章 `sec_01_overview`（第1章执行摘要）标记为 `dirty`，而将 `sec_02_financial`（第2章）保持为 `clean`（锁定状态）。

3. **局部重算 + 全量拼装输出**：Agent **仅读取并重写标记为 `dirty` 的章节**，消除原有的 【待补充材料】 占位符；随后调用 `assemble_full_report()`，将未改动的第2章缓存与新生成的第1、3章合并，直接输出最新版的**完整长篇报告**。

3. 指定章节定点修改（Targeted Section Revision）

当用户提出局部修改指令（例如："把第三章的薪酬竞争力分析改用分位值对比法，并增加研发岗位的单独拆解，其他章节不动"）：

1. **锁定目标章节上下文**：Agent 调用 `get_section_context(section_id="sec_03")`，仅加载第3章的旧版全文、关联的研发岗薪酬数据切片及修改指令。

2. **判断是否触发摘要联动**：如果用户的修改改变了核心结论，状态机提示顺带微调"执行摘要章"；如果只是表述方式或局部格式调整，则仅更新第3章。

3. **全量无损输出**：调用 `commit_section("sec_03", new_content)` 后立即执行 `assemble_full_report()`，输出包含最新第3章的完整长报告，确保其余章节一字不差。

四、 外部工具与 Skill 交付接口设计（适配三大平台）

为了让同一套工程无缝运行在 **Claude Code**、**Codex**、**DeepSeek Harness** 上，工具层建议采用 **"核心 Python 函数库 + CLI 命令行封装 + FastMCP 封装"** 的三合一模式。

1. 五个核心原子工具定义（MCP Tools / CLI Subcommands）

无论底层通过 MCP 还是 Bash CLI 调用，只需向 Agent 暴露以下 5 个高内聚工具：

工具名称 / CLI 命令	输入参数	返回给 Agent 的精简结果	设计目的
`report_init`	`scenario_id`, `workspace_path`	场景大纲结构、所需材料清单	初始化状态树，防止 Agent 自由发挥目录结构
`report_ingest`	`file_paths[]`	新增事实摘要、`dirty_sections[]`（受影响章节列表）、`missing_data[]`	将重度文件解析与表格计算卸载到 Python，不占 LLM 上下文
`report_get_context`	`section_id`	该章节专属 Prompt、关联的 `fact_ledger` 切片、当前章节旧文本	实现**上下文切片隔离**，写哪章只喂哪章的数据
`report_commit_section`	`section_id`, `markdown_body`	校验状态（字数、必填项、数据一致性校验结果）、剩余待更新章节	落盘单章内容，并由代码强制校验数字是否与事实库冲突
`report_assemble`	`output_format` (md/docx)	**完整的全量报告文本** + 导出的文件路径 + 本轮变更 Diff 摘要	保证每轮对话最后一步都能输出完整、格式统一的长文档

2. Skill 指令编排设计（`SKILL.md` / `AGENTS.md` 核心逻辑）

在 `.claude/skills/long-report-generator/SKILL.md`（以及根目录的 `AGENTS.md`）中，写入如下强约束 SOP，控制 Agent 的每轮行为：

```
---
name: long-report-generator
description: 通用多场景长文档分析报告生成与多轮迭代引擎（支持预算经营分析、财务风险尽调、人力薪酬分析等）。当用户上传业务文档要求生
---

# 长报告生成与多轮迭代 SOP（强制执行规范）

你是一个严谨的专业分析报告引擎。在处理任何报告生成或修改请求时，**严禁直接在无状态对话中凭空捏造或一次性裸写整篇超长报告**，必须严格

## 第一步：路由与状态检查
1. 检查当前目录是否存在 `.report_workspace/manifest.json`。
2. 若不存在，根据用户意图识别场景（`budget_analysis` / `fin_due_diligence` / `hr_compensation`），调用 `report_init`

## 第二步：文档增量摄入与影响评估
1. 若用户本轮提供了新文档或补充数据，调用 `report_ingest` 解析文档。
2. 若用户要求修改特定章节，将该章节（及受结论影响的摘要章）列入本轮 `target_sections` 队列。
```

```
## 第三步：按章节上下文隔离撰写
遍历 `target_sections`（或 `dirty_sections`）队列中的每一个 `section_id`：
1. 调用 `report_get_context(section_id)` 获取该章节的方法论规则与结构化事实切片。
2. 严格基于事实切片撰写该章节的深度分析内容（所有核心数值必须附带 `[Doc_ID]` 溯源标记；若数据缺失，必须保留 `【待补充材料：具体名
3. 调用 `report_commit_section` 保存该章节。**严禁触碰不在更新队列中的其他章节！**

## 第四步：强制全量报告拼装与交付（核心红线）
1. 在所有目标章节更新完毕后，**必须**调用 `report_assemble` 生成最新版本的完整长报告。
2. **输出要求**：在每轮对话的最终回复中，先简要说明“本轮更新摘要与联动修改章节”，随后**完整输出 `report_assemble` 生成的全量长
```

五、 在三大 Agent 平台上的落地与验证方案

不同 Agent Harness 对上下文长度和工具调用的机制存在差异，集成与验证时需注意以下适配细节：

验证平台	接入方式	”每次输出全量报告“的工程实现技巧	重点验证指标
Claude Code	`.claude/skills/` + `.mcp.json` 注册本地 Python MCP Server	当报告超过 1.5 万字时，让 LLM 逐字复述工具返回值会极慢且易截断。可在 Skill 中指令 Claude Code：调用 `report_assemble` 将全量报告写入 `final_report.md`，并通过 Bash 工具执行 `cat final_report.md` 直接在终端完整打印全文，既满足终端全量可见，又实现 **0 Output Token 消耗**。	验证多轮补充 Excel 后，跨章节引用数据（如摘要章与财务章）的一致性。
OpenAI Codex CLI	项目根目录放置 `AGENTS.md` + `engine/cli.py` 命令行工具	Codex 擅长在沙箱中连续执行 Shell 命令。将 5 个原子工具封装为 `python -m engine.cli <cmd>`，在 `AGENTS.md` 中要求每次任务以 `python -m engine.cli assemble --print-full` 结尾。	验证沙箱环境下 Pandas 预计算脚本对复杂财务/薪酬表格的容错率。
DeepSeek Harness	System Prompt 注入 `SKILL.md` + Function Calling 绑定 Python 工具函数	在 Harness 的外层 Orchestrator（编排器）中做一个后置钩子（Post-Hook）：当检测到 Agent 调用了 `report_assemble` 工具后，直接将工具返回的完整长文档拼接到 Agent 回复流的末尾，彻底绕过模型的 `max_tokens` 输出瓶颈。	验证长上下文下 DeepSeek 对复杂 JSON 事实切片的指令遵循度与溯源准确率。

六、 系统架构与多轮状态流转交互视图

下面的交互式架构图展示了从**场景配置加载、文档解析入库、多轮增量/定点更新到全量长报告拼装**的完整闭环。你可以通过顶部的步骤按钮切换查看**首轮生成、后续材料补充、指定章节修改**三种模式下的组件激活状态与数据流向：
